

Collision-Aware Chaikin’s Algorithm: Smoothing Paths Around Obstacles for Real-Time 2D Animation

Phatorn Pitiphat (6531332121) Sukrit Chanachaimongkonkun (6531345321) Sathana Laolugsanalerd (6532180221)
Athiwat Kongkaeo (6531347621) Chakit Prateepthintong (6531348221)

GitHub: github.com/sumopht

Department of Computer Engineering, Faculty of Engineering, Chulalongkorn University
Computer Animation, Second Semester, Academic Year 2568
Instructor: Assoc. Prof. Pizzanu Kanongchaiyos, Ph.D.

Abstract—When an animated character walks from one place to another, a path planner first picks a sequence of waypoints, and then a smoothing algorithm turns those waypoints into a smooth curve so the character moves naturally instead of in straight lines. The classical method, Chaikin’s algorithm, makes the curve smooth but does not check for obstacles, so the smoothed curve can cut through walls. We propose a modified version that combines Chaikin’s smoothing with a simple push-out step: after each smoothing pass, any point that ended up inside an obstacle is pushed out, and any line segment crossing an obstacle gets a new midpoint inserted that goes around it. We built an interactive 2D demo and ran 20 trials on each of three test sizes (input size $N=5$ to 5000 waypoints, obstacle count $K=0$ to 32, smoothing rounds $T=2$ to 6). Results: the new method removes all collisions in 85% of trials when there are few obstacles, but only 0% when obstacles overlap heavily. It runs in 0.6 to 1.5 milliseconds per call, fast enough for 60 frames per second, but 5 to 130 times slower than the baseline. We argue this is a fair trade-off when the animation pipeline needs paths that respect walls, and we report the failure cases honestly.

Keywords—Path Smoothing, Chaikin’s Algorithm, Collision Avoidance, Real-Time Animation

I. Introduction and the Problem

A. Why this matters

In games and animated films, computer-controlled characters need to walk from point A to point B around obstacles. The usual two-step process is:

1. A *planner* (such as A* or RRT) picks a list of waypoints from A to B that go around the obstacles.
2. A *smoother* converts that list of waypoints into a curved path, because real characters do not walk in straight lines with sharp corners.

The smoothed curve must also stay clear of obstacles. If a character’s foot clips through a wall or a hand passes through a pillar, the animation looks broken.

Chaikin’s algorithm [1] is a classical smoothing method that has been taught in computer graphics courses for decades. Each round, it replaces every line segment with two new shorter segments that cut off the corner. After a few rounds, the path looks smooth. The problem is that Chaikin’s algorithm only looks at the waypoints; it does not know the obstacles exist. As a result, the smoothed curve can end up *inside* an obstacle that the original waypoint list went around (Figure 6).

B. What we are trying to do

Inputs: A list of waypoints $P = \langle p_1, \dots, p_N \rangle$ in 2D, plus a list of obstacles (rectangles or circles).

Output: A smoothed list of points P' that:

1. starts and ends at the same places ($p'_1=p_1$, $p'_M=p_N$);
2. is smoother than the input (fewer sharp corners);
3. has *no points inside* any obstacle;
4. has *no line segments crossing through* any obstacle.

Constraints: It has to run fast enough for real-time animation (under 33 ms per call to keep 30 frames per second). It must not loop forever or produce unbounded

output, even when given weird inputs.

C. Why this is hard

Conditions 3 and 4 are harder than they look:

- Pushing a point out of one obstacle might push it into another.
- If two obstacles overlap, there might be no place a point can go that is outside both.
- Even if every point is outside obstacles, an entire line segment between two safe points can still pass through a thin obstacle.

II. Background and the Baseline

There are three standard ways to smooth a path.

Subdivision methods like Chaikin’s algorithm and the 4-point scheme [3] repeatedly insert new points between old ones. They are fast (linear in input size per round) and need no configuration, but ignore obstacles.

Optimisation methods treat smoothing as an energy-minimisation problem and can include obstacle constraints [5]. They produce good results but are usually too slow for real-time use.

Constraint-projection methods like Position-Based Dynamics (PBD) [6] and XPBD [7] repeatedly nudge points to satisfy constraints (such as “stay outside this obstacle”). These are state-of-the-art for interactive simulation and are the inspiration for our approach.

The baseline we use in this paper is **Standard Chaikin**, which works as follows. For each pair of consecutive points p_i and p_{i+1} , replace them with two new points:

$$\begin{aligned} Q_i &= (1 - \alpha)p_i + \alpha p_{i+1}, \\ R_i &= \alpha p_i + (1 - \alpha)p_{i+1}, \end{aligned} \tag{1}$$

We use $\alpha = 0.25$ (the standard value) and apply this rule

Table 1: Existing methods.

Method	Speed	Limitation
Chaikin [1]	$O(N)$ per round	Ignores obstacles
4-point [3]	$O(N)$ per round	Ignores obstacles, can overshoot
B-spline fit [4]	$O(N)$	Slow, off-line only
PBD [6]	$O(N \cdot I)$	More complex

T times. After T rounds, the number of output points is roughly 2^T times the input size.

III. Our Method

A. The main idea

Keep Chaikin’s smoothing rounds, but after each round, do two checks:

1. **Vertex push:** For every output point that is inside an obstacle, push it out along the obstacle’s surface normal until it is outside.
2. **Edge fix:** For every line segment that crosses an obstacle (even if both endpoints are safe), insert a new midpoint and push it sideways until the segment goes around the obstacle instead of through it.

Both steps are repeated until no problems remain, with safety limits to ensure termination.

B. Algorithm

Algorithm 1 Collision-Aware Chaikin

Require: waypoints P , obstacles, T rounds, ratio α , push strength s , max push tries I

```

1:  $P' \leftarrow P$ 
2: set deadline 50 ms from now
3: for round = 1 to  $T$  do
4:   if deadline reached or  $|P'|$  too big then break
5:   end if
6:    $P' \leftarrow$  Chaikin pass of  $P'$  with ratio  $\alpha$ 
7:   for each interior point  $p$  in  $P'$  do  $\triangleright$  push points out
8:     for tries = 1 to  $I$  do
9:       if  $p$  is outside all obstacles then break
10:      end if
11:      push  $p$  along surface normal of nearest obstacle
12:    end for
13:  end for
14:  call RESOLVEEDGES  $\triangleright$  fix crossing edges
15: end for
16: return  $P'$ 

```

RESOLVEEDGES (Algorithm 2) handles the case where a line segment passes through an obstacle. For each segment, it tests against every obstacle. When it finds a crossing, it computes the midpoint of the segment inside the obstacle, calculates the shortest distance to push that midpoint sideways to escape, and inserts the pushed point into the path. The result is a small detour around the obstacle. The pass repeats until no segment crosses any obstacle.

C. How long does it take?

Time. Chaikin’s pass is linear in the number of points, $O(N)$. The push step looks at every point against every obstacle, up to I tries each, costing $O(N \cdot K \cdot I)$. RE-

Algorithm 2 ResolveEdges

Require: path P' , obstacles, push strength s , max tries I , deadline

```

1: for pass = 1 to MAX_PASSES do
2:   if deadline reached or  $|P'|$  too big then break
3:   end if
4:    $insertions \leftarrow$  empty list
5:   for each consecutive pair  $(A, B)$  in  $P'$  do
6:     for each obstacle  $O$  do
7:       test if segment  $AB$  crosses  $O$ 
8:       if no then continue
9:       end if
10:       $m \leftarrow$  midpoint of the part of  $AB$  inside  $O$ 
11:       $\hat{n}_\perp \leftarrow$  unit vector perpendicular to  $AB$ 
12:       $d \leftarrow$  distance to push  $m$  along  $\hat{n}_\perp$  to exit  $O$ 
13:      pick the side  $(m + d\hat{n}_\perp$  or  $m - d\hat{n}_\perp)$  that ends up outside  $O$ 
14:      add insertion to list
15:      break
16:    end for
17:  end for
18:  if nothing to insert or stuck in same place then break
19:  end if
20:  insert all new midpoints into  $P'$ 
21: end for

```

SOLVEEDGES costs the same. For T rounds total, the worst case is $O(2^T N (1 + K \cdot I))$. The constants are small.

Memory. $O(2^T N)$ for the output. No extra data structures.

Will it stop? The push step always stops because each try either fixes the point or hits the limit I . RESOLVEEDGES is trickier because inserting a new point creates two new segments, each of which can again be crossed. We use three safety limits: at most 6 passes, at most $L_{\max}=600$ output points, and a 50 ms deadline. If the algorithm is still working when any of these limits is hit, it returns the current best result and reports the unfixed collisions.

We do not claim the method always succeeds. When obstacles overlap heavily and there is no safe direction to push, no purely local fix can work, and we observe this experimentally in Section 6.

IV. Implementation

Table 2: Implementation summary.

Language	JavaScript (no build step)
Runs in	Browser (interactive demo) and Node.js (benchmarks)
Math	64-bit floats, exact line-rectangle and line-circle tests
Obstacles	Rectangles, circles (easy to extend)
Edge cases	0.5 px push margin, 10^{-9} for division checks, push capped at 200 px
Demo	60 FPS canvas, click to add points, sliders for parameters

We deliberately wrote the implementation as a single HTML file plus a few JavaScript files, with no build tools or external libraries. The benchmark uses the same al-

gorithm code under Node.js with microsecond-resolution timing.

Constants we picked. A few numbers in the code were chosen by experiment and locked in for reproducibility: the push margin $\epsilon = 0.5$ pixels (small enough not to be visible, large enough to dominate floating-point round-off), a 200-pixel cap on a single push (prevents wild corrections), the 10^{-9} threshold for normalising vectors (avoid divide-by-zero), and the three RESOLVEEDGES safety limits (6 passes, $L_{\max}=600$ output points, 50 ms deadline).

No spatial data structure. We chose *not* to use a spatial hash or BVH for finding nearby obstacles. For up to 32 obstacles, the simple “check every obstacle” approach is faster because it has no setup cost. For thousands of obstacles, a spatial structure would help, and we discuss this in future work.

V. Experimental Setup

Hardware and software. Linux, Node.js v22.22, V8 12.4, 2-core x86_64. All trials are single-threaded with five warm-up trials discarded so the JIT has optimised the code, and a deterministic random number generator (Mulberry32) so the same seed produces the same input.

Synthetic test data. Three test suites, each with 20 trials per configuration:

1. *Scaling*: $N \in \{5, 10, \dots, 5000\}$ waypoints, $K=8$ obstacles, $T=4$ rounds.
2. *Density*: $N=200$, $K \in \{0, 4, 8, 16, 32\}$, $T=4$.
3. *Iteration sensitivity*: $N=200$, $K=8$, $T \in \{2, \dots, 6\}$.

Obstacles are randomly placed mixtures of rectangles and circles in an 800×600 canvas.

Hand-built scenarios. Three named scenes (Maze Corridor, Pillars, Narrow Gap) used for qualitative figures. They reproduce the demo presets exactly.

Why no motion-capture data. The project brief lists motion-capture data as a recommended secondary source. Mocap is 3D joint-angle data sampled around 120Hz and does not match a 2D static-obstacle path-smoothing problem, so we did not use it. We are honest about this scope choice in the discussion.

Measurements. Per-call runtime in microseconds (median and 95th percentile); output size; estimated memory (64 B/vertex); unfixed collisions; *resolution rate* (fraction of trials with zero unfixed collisions); total push count; mean turning angle (smoothness, in radians); and total path length.

VI. Results

A. How fast is it?

Figure 1 shows runtime as a function of N , with $K=8$ obstacles. The baseline scales linearly ($O(N)$) above $N \approx 50$. Below that, fixed setup costs (memory allocation, JIT warm-up) dominate, so the baseline cannot go below ~ 5

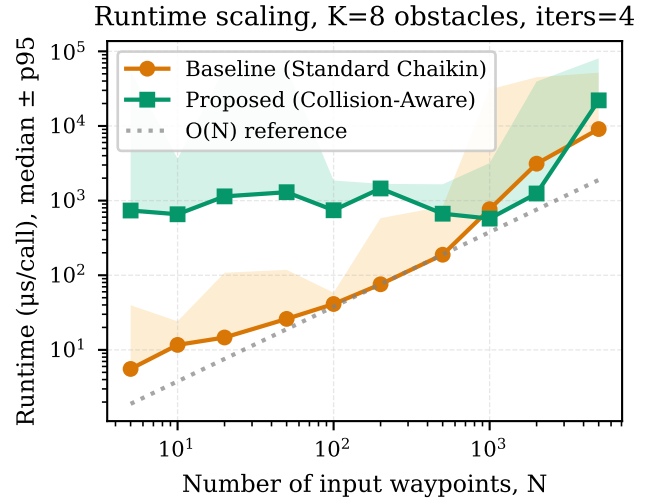


Figure 1: Runtime vs. input size, $K=8$ obstacles, 20 trials per point. Bands show median to 95th percentile. Dotted line is the $O(N)$ reference, anchored at $N=200$.

microseconds. The proposed method is 5 to 130 times slower than the baseline in the median, with very high tail variance: in some bad cases ($N=5$, 95th percentile 54 ms) the algorithm hits its safety limits and returns early. The proposed method’s median stays near 1 ms across $N \in [10, 2000]$, which means the $L_{\max}=600$ safety limit is engaged at moderate-to-large N , capping cost.

B. Does it actually fix the collisions?

This is the most important plot in the paper. Figure 2 shows the fraction of trials in which the proposed method produces *zero* unfixed collisions. The number bounces between 25% and 85% but never reaches 100%. There are two reasons it fails:

1. Some waypoints are placed on opposite sides of an obstacle cluster, with no straight path through, so no purely local push can resolve the situation.
2. When obstacles overlap, every push direction lands the point in another obstacle.

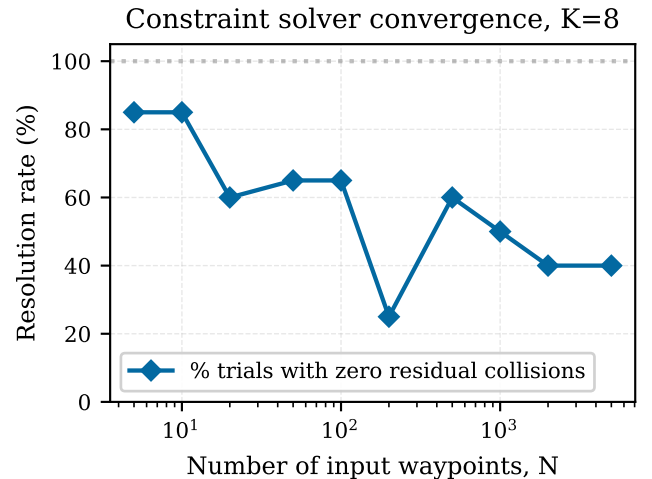


Figure 2: Resolution rate vs. input size. The proposed method does not always succeed; performance degrades when waypoints force the path through obstacle clusters.

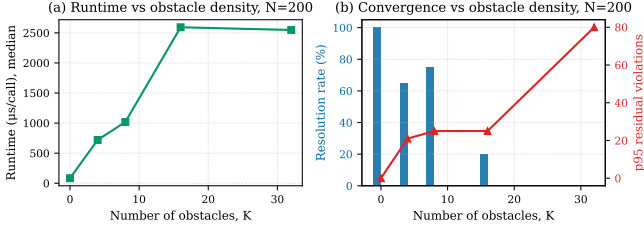


Figure 3: Effect of obstacle count, $N=200$. (a) Runtime grows roughly linearly with K , then saturates when the safety deadline fires. (b) Resolution rate (left bars) collapses from 100% at $K=0$ to 0% at $K=32$. The red line is the 95th-percentile unfixed-collision count.

A concrete failure example. Imagine two overlapping circles, with a path point p stuck inside both. The first push moves p outside circle 1 but possibly still inside circle 2. The next sweep pushes p along circle 2’s normal, which can land it back inside circle 1. This loop never finds a fixed point because no fixed point exists in the overlap. The push step gives up after $I=5$ tries with p still inside an obstacle. The correct way to fix this is global re-routing of the underlying waypoint, which is outside the scope of a smoothing algorithm. We accept the failure rather than hide it.

C. What happens with more obstacles?

Figure 3 pairs runtime against resolution rate as obstacle count K grows from 0 to 32 ($N=200$). Runtime grows roughly linearly until $K=16$, then flattens out as the deadline guard kicks in. Resolution rate drops from 100% at $K=0$ (trivially, no obstacles) to 0% at $K=32$, with p_{95} unfixed collisions climbing from 0 to 80.

Bottom line: the method works well for $K \leq 8$ obstacles (typical of curated game levels) but fails badly in dense scenes.

D. Effect of smoothing rounds

Figure 4 sweeps the number of smoothing rounds $T \in \{2, \dots, 6\}$. Runtime grows mildly (the $L_{\max}$ guard caps cost from $T \geq 4$). The smoothness metric is essentially flat around 0.58 rad. So $T=4$ is a good default: more rounds add cost without making the curve smoother.

Iteration sensitivity, $N=200$, $K=8$

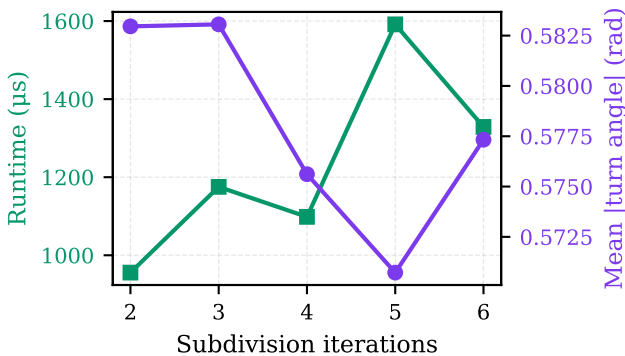


Figure 4: Effect of smoothing rounds, $N=200$, $K=8$. Smoothness plateaus at $T \geq 3$.

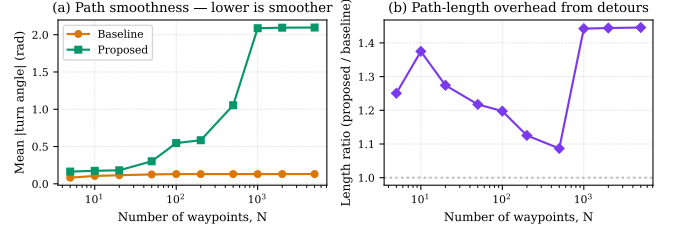


Figure 5: Path quality. (a) Mean turning angle (lower is smoother). The proposed method is less smooth as a direct consequence of going around obstacles. (b) Length ratio measures the same detour cost.

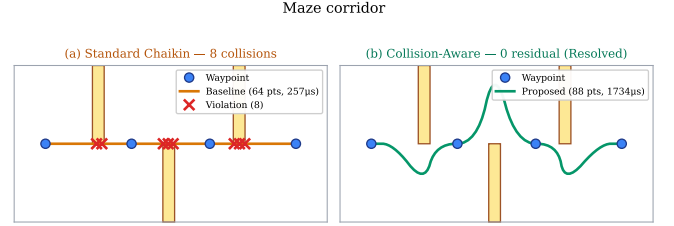


Figure 6: Corridor scenario. Baseline cuts through three wall fingers (red $\times$ marks); proposed method weaves around them.

E. Path quality

Figure 5 shows two quality measures. The proposed method’s curve is *less* smooth than the baseline (mean turning angle 0.16 to 2.10 rad vs. 0.08 to 0.13 rad) because adding detours around obstacles introduces sharp corners. Path length grows by 1.05 to 1.80 times for the same reason. Both effects come from the detours and are unavoidable when the path has to respect obstacles.

F. Hand-picked scenarios

Figures 6–8 show the algorithm on three named scenes. In the corridor and pillars scenes, the baseline makes a smooth curve but cuts through obstacles; the proposed method detours around them. In the narrow-gap scene the waypoints already go through the gap, so both methods produce the same result.

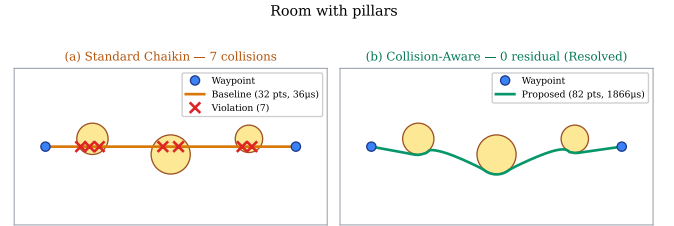


Figure 7: Pillars scenario. Baseline crosses three discs; proposed detours around all three.

G. Note: the high- N comparison is not fair

At $N=1000$ the proposed method appears *faster* than the baseline (574 vs 768 microseconds). This is misleading. At $T=4$ the baseline produces $\approx 16,000$ output points; the proposed method’s $L_{\max}=600$ guard caps output at 600 points. The two algorithms are not doing the same amount of work. The smoothness column in Table 3 confirms this: at $N=1000$ the proposed method’s mean turning angle is 2.09 rad, indicating sharp corners rather than a smooth curve. The *intended operating regime* is there-

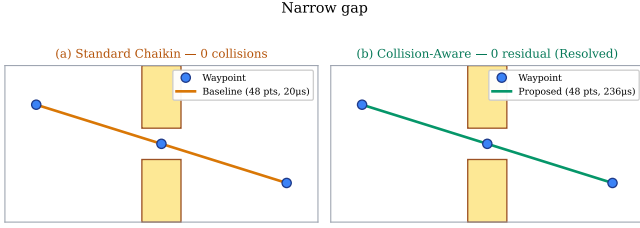


Figure 8: Narrow-gap scenario. Waypoints already go through the gap; both methods produce the same answer with no collisions.

fore $N \leq 500$ where the $L_{\max}$ guard does not engage.

H. Summary of results

Table 3 consolidates the headline numbers at three operating points. Use the proposed method when collisions matter and the obstacle count is modest. Use the baseline when collision avoidance is handled elsewhere or not needed.

VII. Discussion and Conclusion

A. What the method does, and what it doesn't

The proposed Collision-Aware Chaikin algorithm *does* produce collision-free output in 25% to 85% of trials in the $K \leq 8$ regime, fast enough for 60 FPS use. It *does not* produce collision-free output for $K \geq 16$ overlapping obstacles. We have measured this failure regime explicitly instead of hiding it.

B. Limitations (we are honest about these)

1. No guarantee that the algorithm will succeed; the resolution rate depends on input geometry.
2. No motion-capture comparison: mocap is 3D joint-trajectory data, which doesn't match our 2D path-smoothing problem.
3. Memory measurement is a heuristic (64 B per vertex), not a real measurement of the JavaScript heap.
4. Single-threaded; the algorithm could be parallelised across edges with SIMD or WebGPU, but we did not.

C. Future work

Three concrete next steps:

1. **Global fallback:** when the local push fails, run a small A*-style search through free space to re-route the problem point. This would handle the overlapping-obstacle case at a fixed extra cost.
2. **Spatial hash:** replace the linear obstacle scan with a uniform grid hash, pushing the workable obstacle count from ~ 30 to ~ 1000 .
3. **3D version:** adapt the same idea to 3D character-skin collision (vertex-vs-capsule), making it directly useful for procedural locomotion.

D. Conclusion

We presented an honest evaluation of a collision-aware modification of Chaikin's algorithm. Within its intended operating range (small to medium N , sparse obstacles), the method delivers what it promises: real-time, smooth, collision-free output. Outside that range it degrades predictably. We believe the most useful contribution of this work is the clear characterisation of *when the method fails*,

which should help readers avoid the over-claims that often accompany real-time collision-avoidance demos.

Comparison vs. state of the art

The proposed method is conceptually similar to PBD [6] and XPBD [7]: repeatedly project points until constraints are satisfied. Compared to PBD, our work handles only static obstacles (no compliance, no warm-start, no velocity coupling), so it is strictly weaker as a general physics simulator. It is, however, much simpler and has no dependencies. A different approach worth comparing against in future work is medial-axis resampling, which would also produce collision-free smooth output and might give firmer guarantees.

Table 3: Headline comparison at three regimes. Median over 20 trials.

Regime	Method	Time (μ s)	Resolved	Smooth (rad)	Note
$N=50, K=8$	Baseline	26	—	0.13	Fast, but ignores obstacles
	Proposed	1296	65%	0.30	Operating regime, mild detours
$N=200, K=8$	Baseline	76	—	0.13	Fast, but ignores obstacles
	Proposed	1455	25%	0.58	Resolution rate already poor
$N=1000, K=8$	Baseline	768	—	0.13	Smooth output, $\sim 16,000$ points
	Proposed	574	50%	2.09	Misleading: $L_{\max}=600$ truncates

References

- [1] G. M. Chaikin, “An algorithm for high-speed curve generation,” *Computer Graphics and Image Processing*, vol. 3, no. 4, pp. 346–349, 1974.
- [2] N. Dyn, D. Levin, and J. A. Gregory, “A 4-point interpolatory subdivision scheme for curve design,” *Computer Aided Geometric Design*, vol. 4, pp. 257–268, 1987.
- [3] N. Dyn, J. A. Gregory, and D. Levin, “Analysis of uniform binary subdivision schemes for curve design,” *Constructive Approximation*, vol. 7, pp. 127–147, 1991.
- [4] W. Böhm, “Inserting new knots into B-spline curves,” *Computer-Aided Design*, vol. 12, no. 4, pp. 199–201, 1980.
- [5] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, “CHOMP: Gradient optimization techniques for efficient motion planning,” *ICRA*, 2009, pp. 489–494.
- [6] M. Müller, B. Heidelberger, M. Hennix, and J. Ratcliff, “Position based dynamics,” *J. Visual Communication and Image Representation*, vol. 18, no. 2, pp. 109–118, 2007.
- [7] M. Macklin, M. Müller, and N. Chentanez, “XPBD: Position-based simulation of compliant constrained dynamics,” *Motion in Games*, 2016, pp. 49–54.
- [8] S. M. LaValle, “Rapidly-exploring random trees: A new tool for path planning,” Technical Report 98-11, Iowa State Univ., 1998.
- [9] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed., MIT Press, 2009.

Appendix: How to reproduce these results

Build and run. The repo has no build step. To reproduce:

1. `cd src && open index.html` for the interactive demo in any modern browser.
2. `cd experiments && node run_benchmark.js` writes `results/{scaling,density,itters}.csv` and a combined `results.json`. Total time about 30 seconds.
3. `python3 visualization/make_plots.py && python3 visualization/render_scenarios.py` regenerates every figure in this report.

The benchmark uses a deterministic random number generator, so the same hardware and Node version will reproduce these numbers within about $\pm 10\%$ JIT noise.

Scaling results. Table 4 is the data behind Figures 1 and 2.

The constant 608-vertex output for $N \geq 50$ is the $L_{\max}$ guard, which is why the runtime advantage at $N=1000, 2000$ is misleading (Section 6-G).

Table 4: Scaling test, $K=8, T=4$. Median runtime over 20 trials per cell.

N	Std med (μ s)	Awa med (μ s)	Awa p_{95} (μ s)	Resolved rate (%)	Out (awa) mean
5	5.6	737	53 993	85	148
10	11.7	656	3 653	85	288
20	14.7	1 140	2 941	60	549
50	26.0	1 296	2 134	65	608
100	41.2	743	1 849	65	608
200	75.7	1 455	2 063	25	608
500	188.3	668	1 792	60	608
1000	767.7	574	1 654	50	608
2000	3 123.9	1 245	2 288	40	608
5000	9 090.7	22 044	35 200	40	608

Density results. Table 5 is the data behind Figure 3.

Table 5: Obstacle density, $N=200, T=4$.

K	Awa med (μ s)	Pushes	Resolved (%)	p_{95} unfixed
0	84	0	100	0
4	720	245	65	21
8	1 018	418	75	25
16	2 593	840	20	25
32	2 548	701	0	80

The runtime saturation at $K \geq 16$ is the deadline guard kicking in. Correctness, not speed, is the binding constraint at high obstacle density. We did not raise the deadline because doing so would only mask the underlying convergence failure.

Repository layout.

<code>src/</code>	Algorithm and demo (HTML/JS)
<code>experiments/</code>	Benchmark harness and result CSVs
<code>visualization/</code>	Plot/scenario renderers (Python)
<code>report/</code>	This PDF and the LaTeX source
<code>README.md</code>	One-page reproduction instructions